

Examen Parcial: Estructuras de Datos en la Exploración Espacial

Instrucciones Generales

- Este examen está diseñado para evaluar su comprensión y aplicación de los conceptos de estructuras de datos, incluyendo TADs, listas enlazadas, pilas, filas, árboles, árboles binarios y árboles binarios de búsqueda.
- Responda cada ejercicio con claridad y justifique cuando sea indicado.
- El examen se basa en la exploración espacial, utilice su creatividad para implementar lo pedido.

Ejercicio 1: Implementación de TADs

Enunciado

En esta simulación de exploración espacial, cada nave tiene una tripulación de astronautas. Para este ejercicio deberá implementar dos clases en Python: **Astronauta** y **NaveEspacial**.

Clases a Implementar

1. Clase **Astronauta**:

◦ **Atributos:**

- **nombre**: (*str*) Nombre del astronauta.
- **especialidad**: (*str*) Especialidad del astronauta (por ejemplo, Piloto, Ingeniero, Médico).
- **experiencia**: (*int*) Años de experiencia del astronauta (debe estar entre 0 y 50).

◦ **Métodos:**

- **__init__()**: Constructor que inicializa los atributos del astronauta.
- **aumentar_experiencia()**: Método que incrementa la experiencia del astronauta en 1 año, siempre y cuando no supere los 50 años.
- **__str__()**: Método que devuelve una representación en cadena del astronauta en el formato: "Nombre: [nombre], Especialidad: [especialidad], Experiencia: [experiencia] años"

2. Clase **NaveEspacial**:

◦ **Atributos:**

- **nombre**: (*str*) Nombre de la nave espacial.
- **tripulacion**: (*lista*) Lista que contiene instancias de la clase **Astronauta**.

◦ **Métodos:**

- **__init__()**: Constructor que inicializa el nombre de la nave y crea una lista vacía para la tripulación.
- **agregar_astronauta()**: Método que agrega un astronauta a la tripulación. Si ya hay 7 astronautas en la tripulación, debe mostrar un mensaje indicando que no se pueden tener más astronautas.

- `mostrar_tripulacion()`: Método que imprime todos los astronautas de la tripulación utilizando el método `__str__` de la clase `Astronauta`.
- `experiencia_promedio()`: Método que calcula y devuelve la experiencia promedio de los astronautas en la tripulación. Si no hay astronautas en la tripulación, debe devolver 0.

Ejercicio 2: Recursividad en la Exploración Espacial

Enunciado

Los exploradores espaciales a menudo se enfrentan a desafíos que requieren la búsqueda de planetas habitables en un sistema estelar. Para este ejercicio implementar una función recursiva que simula la búsqueda de un planeta específico en un árbol de cuerpos celestes.

Contexto

Suponga que tiene un árbol binario donde cada nodo representa un cuerpo celeste, cada nodo tiene un nombre y puede tener hasta dos hijos (uno a la izquierda y otro a la derecha). La búsqueda de un planeta específico se puede realizar de manera recursiva verificando si el planeta se encuentra en el nodo actual o en uno de sus subárboles.

Clases a Implementar

1. Clase `CuerpoCelesteNode`:

◦ Atributos:

- `nombre`: (*str*) Nombre del cuerpo celeste.
- `izquierda`: (*CuerpoCelesteNode*) Hijo izquierdo (puede ser `None`).
- `derecha`: (*CuerpoCelesteNode*) Hijo derecho (puede ser `None`).

◦ Métodos:

- `__init__(self, nombre: str)`: Constructor que inicializa el nombre del cuerpo celeste y establece los hijos izquierdo y derecho como `None`.

2. Función Recursiva `buscar_planeta`:

◦ Parámetros:

- `nodo`: (*CuerpoCelesteNode*) Nodo actual del árbol donde se realiza la búsqueda.
- `nombre_planeta`: (*str*) Nombre del planeta que se busca.

◦ Retorno:

- Devuelve `True` si el planeta es encontrado en el árbol, y `False` si no se encuentra.

◦ Lógica:

- Si el nodo actual es `None`, devuelve `False`.
- Si el nombre del nodo actual coincide con `nombre_planeta`, devuelve `True`.
- Llama recursivamente a sí misma para buscar en el hijo izquierdo y luego en el hijo derecho.

Ejercicio 3: Sistema de Órbitas Planetarias

Enunciado

Cada planeta tiene una serie de órbitas que lo rodean, para gestionarlas implementar un sistema utilizando una **pila** (stack), que permita llevar un registro de las órbitas en orden inverso ya que la más cercana al planeta es la que se estudiará primero.

Contexto

La pila se utilizará para almacenar las órbitas de un planeta. Cuando se descubre una nueva órbita, se agrega a la pila. Si los científicos deciden estudiar una órbita, se puede desapilar la órbita más cercana.

Clases a Implementar

1. Clase **Orbita**:

- **Atributos:**
 - **nombre**: (str) Nombre o designación de la órbita.
- **Métodos:**
 - **__init__(self, nombre: str)**: Constructor que inicializa el nombre de la órbita.

2. Clase **PilaOrbitas**:

- **Atributos:**
 - **orbitas**: (lista_enlazada) pila que almacena las órbitas en orden.
- **Métodos:**
 - **__init__(self)**: Constructor que inicializa la lista de órbitas como vacía.
 - **apilar(self, orbita: Orbita)**: Método que agrega una órbita a la pila.
 - **desapilar(self)**: Método que elimina y devuelve la última órbita agregada a la pila. Si la pila está vacía, debe devolver un mensaje indicando que no hay órbitas para desapilar.
 - **mostrar_orbitas(self)**: Método que imprime todas las órbitas en el orden en que fueron apiladas.

Ejercicio 4: Diseño Teórico de un Juego de Exploración Espacial

Enunciado

Imagine que debe desarrollar un juego de exploración espacial.

Este juego incluirá elementos como el descubrimiento de planetas, misiones espaciales, estaciones espaciales, recolección de recursos y encuentros con civilizaciones alienígenas. Para cada uno de estos componentes deberá explicar, qué estructuras de datos utilizaría y por qué; considerando la eficiencia y eficacia en cada caso.

Componentes mínimos del Juego a considerar

1. **Descubrimiento de Planetas**
2. **Misiones Espaciales**
3. **Estaciones Espaciales y Colonias**
4. **Navegación Interestelar**
5. **Encuentros con Civilizaciones Alienígenas**

Consideraciones Finales

Al diseñar este juego de exploración espacial es fundamental, considerar no solo, la estructura de datos más adecuada para cada componente sino también cómo estas estructuras interactúan entre sí. La elección de estructuras eficientes además de mejorar el rendimiento del juego también proporciona una mejor experiencia al jugador al permitir interacciones fluidas y rápidas.

Criterios de Evaluación

1. Corrección y Complejidad del Código (30 puntos)

- Implementación correcta de las clases y métodos solicitados.
- Uso adecuado de las estructuras de datos.
- Complejidad algorítmica y eficiencia del código.

2. Claridad y Legibilidad (20 puntos)

- Estructura y organización del código.
- Uso de comentarios que expliquen la lógica y el propósito del código.
- Nombres de variables y métodos descriptivos.

3. Justificación Teórica (40 puntos)

- Explicación clara y lógica de las elecciones de estructuras de datos en el ejercicio teórico.
- Consideración de la eficiencia y eficacia en el uso de las estructuras seleccionadas.
- Integración creativa y coherente de los elementos de la exploración espacial en el diseño del juego.

4. Manejo de Casos Especiales (10 puntos)

- Capacidad para manejar errores y condiciones especiales en la implementación.
- Robustez del código frente a entradas inesperadas o situaciones límite.

Puntaje Total Exploración Espacial: 100 estrellas

Dato

Los estudiantes disponen de **4 horas** para resolver el examen. Se recomienda gestionar bien el tiempo para abordar cada ejercicio con la profundidad necesaria.

¡Buena suerte, exploradores espaciales!